発表シミュレーション — IntSeqBERT @ CICM 2026

- 想定: 30分枠（発表25分 + 質疑5分）、話速 ~100 wpm（ゆっくりめの英語）
- 各スライド: 想定時間 / 話す内容（ほぼスクリプト・短文中心） / 次スライドへの **Bridge**（つなぎの一文）
- ⚠️ = ストーリーが詰まる・流れが切れるリスクのある箇所
- 末尾に総評と改善提案

Slide 1: Title — 0:00–0:45

> Good morning, everyone. I am Kazuhisa Nakasho, from Iwate Prefectural University, Japan. Today I will talk about IntSeqBERT. It is a Transformer encoder that learns *arithmetic structure* from integer sequences in the OEIS. All the code and data are already public on GitHub — you can try everything I show today.

Bridge: "Let me start with the big picture: why integer sequences, and why now."

Slide 2: Long-Term Goal — 0:45–3:00

> In the history of mathematics, many deep conjectures started from a *numerical observation*. A famous example: someone noticed that 196,884 — a coefficient of the j-function — is 196,883 plus 1, a dimension from the Monster group. That observation became Monstrous Moonshine. The BSD conjecture and mirror symmetry also started from numerical patterns.
>
> Today, AI systems like AlphaProof, the Ramanujan Machine, and FunSearch show that machines can take part in this kind of discovery. Our long-term goal is in the same spirit: an AI that finds *hidden number-theoretic similarities* between integer sequences from completely different fields — and helps us generate conjectures.
>
> The natural data source is the OEIS: almost four hundred thousand sequences, each with a precise mathematical definition. A uniquely machine-readable corpus of mathematics.
>
> But one piece is missing. To compare sequences, a machine needs a *representation* that captures number-theoretic structure. This paper is the first step. We propose such a representation — and, importantly, we *measure* how much arithmetic structure it really captures.

Bridge: "So why is this difficult? The reason is a fundamental weakness of deep learning."

> 意図メモ: 「予測精度を競う論文ではなく、表現を測る論文」というフレームを開始3分で確立。 R1対応の核。Moonshineの具体的な数（196,884 = 196,883 + 1）をフックとして冒頭に置く。

Slide 3: Deep Learning Cannot Multiply — 3:00–5:00

Here is the problem in one sentence: **deep learning cannot multiply.** Neural networks have a well-known *spectral bias*: they learn low-frequency, addition-like structure first. Multiplicative laws — n squared, factorial, exponential growth — emerge only after *many layers* of composition. Sometimes they never emerge at all.

Standard tokenization makes this worse. A fixed vocabulary cannot represent astronomically large values — they all collapse into a single UNK token. And whatever arithmetic structure exists is buried inside opaque token IDs.

And the OEIS is the *worst case* for this weakness. One single corpus contains single-digit constants — and values around ten to the 210.

Bridge: "Our key idea to attack this problem is not a new neural trick. It is classical number theory."

意図メモ: ユーザー強調ポイント①。タイトルを断言形にして問題を一文で刻む。 口頭で grokking（modular arithmetic の創発に長い訓練が必要）に触れてもよい。

Slide 4: Modular Arithmetic Makes Multiplication Shallow — 5:00–7:00

Reduction modulo m is a **ring homomorphism** from the integers to Z mod m. That means it preserves *both* addition *and* multiplication. So the residue sequence — x mod m — keeps the multiplicative structure of the original sequence. And it does so *regardless of magnitude*. Ten to the 210? The residue is still just a number between 0 and m minus 1.

Number theory gives us periodicity for free. Fermat's little theorem makes powers periodic modulo a prime. Quadratic reciprocity links residues across different primes. And the Chinese Remainder Theorem means composite moduli aggregate several prime powers at once.

So here is our hypothesis: if we give the network the *modulo spectrum* as explicit input, it can see multiplicative structure **directly — in the input layer, at depth zero** — instead of spending many layers to rediscover it.

Bridge: "To test this hypothesis, we need a concrete task. Here it is."

意図メモ: ユーザー強調ポイント②。Slide 3「深い層が要る」⇔ Slide 4「深さゼロで見える」の対比が このトークの背骨。"at depth zero" は口頭で強調する。

Slide 5: Task — Masked Sequence Modelling — 7:00–8:30

The task is masked sequence modelling — BERT-style, but for integers. We take a prefix of an OEIS sequence, up to 128 terms, and mask 15 percent of the positions. The model must predict each masked value from the surrounding context. This directly probes how well the model has internalised

the law of the sequence. Next-term prediction is the special case where the last term is masked.

The prior benchmark FACT, from NeurIPS 2022, studied this with a plain Transformer and found unmasking to be the hardest task. Our Vanilla baseline corresponds to that architecture.

Now — instead of predicting one token, we predict *three* quantities per masked position: the magnitude on a log scale, the sign, and the residues modulo every m from 2 to 101.

Bridge: "These three targets correspond directly to the two streams of our architecture."

Slide 6: Dual-Stream Architecture — 8:30–10:30

Here is IntSeqBERT. [図を指しながら] The **magnitude stream** encodes the log-scale size and the sign — four numbers. The **modulo stream** encodes, for each modulus m from 2 to 101, the residue as a point on the unit circle — sine and cosine. You may recognise this: it is the Transformer's sinusoidal *positional* encoding — but we repurpose it for *residues of values*. The circle respects the cyclic group structure of Z mod m: residue m minus 1 and residue 0 are neighbours, with no discontinuity.

The two streams are fused by FiLM: the modulo embedding produces a scale and a shift that modulate the magnitude embedding, element-wise. Then a standard pre-LN Transformer encoder. Three sizes — the largest is 91.5 million parameters.

Bridge: "Training is multi-task: one loss per prediction target." （→Slide 7は軽く流す）

Slide 7: Multi-Task Training — 10:30–11:30

Three heads on top of the encoder. The magnitude head also predicts its own *uncertainty* — remember this, the Solver will use it. The modulo head is 100 independent classifiers, one per modulus, each normalised by its maximum entropy so that mod 2 and mod 101 contribute fairly.

We train on about 220,000 OEIS sequences for 200 epochs — on a *single consumer GPU* with 8 gigabytes of memory. Everything I show today is reproducible on a gaming PC.

Bridge: "But the network outputs distributions, not integers. To get an actual integer, we need one more component."

意図メモ: ⚠️ここは1分で流す。数式の読み上げはしない。 "uncertainty — remember this" で Slide 8 への伏線を張る。

Slide 8: The CRT Solver — 11:30–13:00

The network gives us a magnitude distribution, a sign, and 100 residue distributions. The **Solver** turns them into a concrete integer — *symbolically*. From the magnitude and its uncertainty, we get a three-sigma search interval. If the interval is small, we simply enumerate. If it is large, we run a CRT-guided beam search anchored on the most confident moduli. For very large integers, sparse CRT generates candidates directly.

I want to emphasise the design philosophy here. This is a **neural-symbolic** system. Residue arithmetic and the Chinese Remainder Theorem are *built in* — we do not ask the network to discover number theory end-to-end. Whether that is a feature or a limitation — I will come back to this question with the results.

Bridge: "Before the results, two quick slides: the data and the metrics."

意図メモ: "Whether that is a feature or a limitation…" は Slide 16 への明示的な伏線。 R2懸念（手作り特徴量への疑義）を自分から先に口に出すことで、防御ではなく論点として提示する。

Slide 9: Dataset — 13:00–14:00

From 391,710 OEIS entries we keep 274,705 — about 70 percent. We exclude eleven tags, in three groups. First, representation-dependent sequences — decimal expansions, continued fractions — where the structure lives in the *encoding*, not in the integer values. Second, tags incompatible with our task setting, like tables. Third, quality labels. I admit some of these are debatable — "less" and "dumb" are subjective labels, and tables could be linearised. This is a practical choice for a first benchmark.

For evaluation we stratify values into five magnitude buckets, from Small — below 100 — up to Astronomical — beyond ten to the fifty.

Bridge: "One metric needs explanation before you see the numbers."

意図メモ: R3対応。「異論は認める」を発表者側から先に言う。1分で切り上げる。

Slide 10: Metrics — What $\text{Acc}_{0.5}$ Means — 14:00–15:00

Magnitude Accuracy — Acc 0.5 — is **not** exact match. It asks: is the prediction within 0.5 on the log scale — a factor of about 3? For example, if the true value is one million, anything between roughly 320,000 and 3.2 million counts. Why so loose? Because its job is only to set a *realistic search range* for the Solver. **Exact** integer match is measured separately — by Solver Top-k, which you will see in a moment. We also use NIG — normalised information gain — to measure how much periodic structure was learned for each modulus. Think of it as: zero means nothing learned, one means perfect.

Bridge: "Now, the results. Three questions: Does the dual stream help? Where does it help? And can we actually predict integers?"

意図メモ: R2対応（Acc0.5の直観）。結果パートを3つの問いで予告し、聞き手に地図を渡す。

Slide 11: Main Results — 15:00–16:30

First question: does it help? Yes, consistently. At the Large scale, IntSeqBERT reaches 95.85 percent magnitude accuracy — nine points above the tokenised baseline — and 50.38 percent mean modulo accuracy.

The ablation row is the important one. Remove the modulo stream, and mean modulo accuracy drops by *fifteen points*. But notice — magnitude accuracy *also* drops, by six points. Knowing the residues constrains what the value can be. The modulo stream is not just answering modulo questions — it is acting as an arithmetic constraint on the whole representation.

Bridge: "Second question: *where* does it help? The answer is: exactly where tokenization breaks."

Slide 12: Where the Gains Come From — 16:30–18:00

This scatter plot is predicted versus true magnitude. [図を指しながら] Look at the Vanilla model: above the vocabulary limit, everything collapses into UNK, and the predictions scatter. In the Large bucket its error is *thirteen times* ours.

One honest caveat. Most OEIS sequences grow slowly, so the *overall* accuracy numbers are flattered by easy cases. We do not hide this. The real contribution shows up in the hard buckets — Large and above — where the dual-stream model keeps working and the baseline does not.

Bridge: "Third question — the strictest test. Can we reconstruct the exact integer?"

意図メモ: R3対応（成長バイアス）を結果の文脈で自分から先に開示。 ⚠️ 現行ドラフトではこの後 Slide 13–14（NIG）に行くが、口頭の流れでは 「3つ目の問い＝厳密一致」と宣言した直後に NIG が挟まると問いが宙に浮く。→ 総評の提案1参照。

Slide 13: Solver Next-Term Prediction — 18:00–19:30

Here is the end-to-end test: predict the exact last term of 10,000 held-out sequences. The tokenised Transformer gets 2.59 percent. IntSeqBERT gets 19.09 percent — a **7.4-fold** improvement.

By bucket: 68 percent for small values, 21 percent for medium. The baseline is at *zero* for anything beyond its vocabulary. But look at the large buckets: our model also drops to near zero. CRT reconstruction needs *exact* residues for many moduli *simultaneously* — and at 50 percent modulo accuracy, that almost never happens.

Bridge: "So — 19 percent. Is that a success or a failure? Let me be honest about this number."

Slide 14: Reading the Results Honestly — 19:30–21:00

Both, actually. And both readings are informative. Even with number-theoretic structure *built into* the representation, exact prediction of OEIS terms remains hard. A pure tokenised Transformer is almost hopeless. So this is also a **negative result** — and I think a useful one: it quantifies the limits of pure end-to-end learning on arithmetic.

Earlier I asked whether handcrafted features are a step backward. Here is my answer: in this study, they are the **measurement instrument**. Because the features are explicit, the ablation tells us *exactly* how much arithmetic structure each representational choice captures — fifteen points of modulo accuracy. And the result is a case *for* neural-symbolic designs: the symbolic CRT solver supplies exact constraints that the network demonstrably cannot learn alone.

Bridge: "Prediction accuracy is one lens. But remember our real goal: representation. What did the network actually *learn* about number theory? This, to me, is the most interesting part."

意図メモ: Slide 8 で張った伏線（feature or limitation?）をここで回収。R2の総評を全面的に受け入れる。次の bridge で「ここからが本番（数論的発見）」と上げ直すのがこのトーク最大の感情曲線。

Slide 15: NIG Spectrum — 21:00–22:15

For every modulus from 2 to 101, we measure the normalised information gain. [図を指しながら] IntSeqBERT is above both baselines across the *entire* spectrum. But look at the shape — it is not flat. The peaks are systematic. The single best modulus, across all models and scales, is 96 — two to the fifth times three. The runner-up is 2 — parity — which makes sense: parity is everywhere in the OEIS.

Bridge: "Why 96? Why composites? It turns out there is one number-theoretic quantity that explains the whole shape."

Slide 16: NIG vs Euler's Totient Ratio — 22:15–23:45

Plot NIG against phi of m over m — Euler's totient ratio — and you get this. [図を指しながら] Correlation minus 0.85, p below ten to the minus 28. The interpretation is the CRT aggregation effect: x mod 96 *simultaneously* encodes x mod 2, 4, 8, 16, 32, and mod 3. Composite moduli with many small prime

factors carry more information per modulus.

This is an empirical observation, not a theorem. But it has a design implication: if you extend the spectrum beyond 101, add *highly composite numbers* — not more primes.

And one more thought. This spectrum is a *fingerprint* of the arithmetic structure of a corpus. Two sequences — or two corpora — can be compared by their spectra.

Bridge: "That fingerprint idea brings me back to where I started: what is this work *for*, in computer mathematics?"

意図メモ: R2対応（NIG-φ相関の含意）。"fingerprint" → Slide 17 → Slide 2 と首尾呼応する。 この並びなら結果パートの最後が「発見」で終わり、negative result が物語の底として機能する。

Slide 17: Positioning in Computer Mathematics — 23:45–24:45

Program synthesis on the OEIS — Alien Coding, QSynt, Learning Conjecturing — describes each sequence's *generation rule*. We do something complementary: we learn *shared arithmetic structure* across the whole corpus. A synthesis system asks: what program makes this sequence? Our representation asks: which sequences are *number-theoretically similar* — even if they are generated in completely different ways? Pretrained embeddings could serve synthesis as a search heuristic, or as a retrieval index. That retrieval capability is the bridge toward conjecture generation — the Ramanujan-Machine spirit I began with.

Bridge: "Let me summarise."

Slide 18: Conclusion — 24:45–25:45

Four takeaways. One: the modulo spectrum is principled feature engineering — a ring homomorphism, which makes multiplicative structure visible at depth zero. Two: it works — consistent gains everywhere, 7.4 times on exact prediction. Three: composite moduli aggregate arithmetic information — the totient correlation. Four: honestly — pure neural prediction of OEIS terms remains hard. The future is neural-symbolic.

Next steps: approximate CRT for large integers, highly composite moduli beyond 101, retraining at larger scale, and sequence-similarity retrieval for conjecture generation. Code and data are on GitHub. Thank you.

総評

流れの評価: 概ね通る。感情曲線に1箇所だけ構造的な問題

強い点

- Slide 3⇔4 の対比（「多層かかる」⇔「深さゼロで見える」）が背骨として機能する。フックも立つ。
- Slide 2 → 17 → 18 の首尾呼応（conjecture generation で開いて閉じる）が R1 対応と一致。
- Slide 8 の「feature or limitation?」→ Slide 16 で回収、という伏線構造が R2 対応と一致。

提案1（最重要）: 結果パートの順序を 11 → 12 → 15 → 16 → 13 → 14 に並び替え

現行順（11,12 → 13,14 NIG → 15 Solver → 16 negative）には2つの問題があります：

1. Slide 10 の bridge で「3つの問い（効くか／どこで効くか／整数を当てられるか)」と予告すると、 3問目（Solver）の前に NIG 分析が挟まり、問いが2枚分宙に浮く。
2. 結果パートが Slide 16（negative result）で終わり、**物語が下げで終わる**。

並び替え後は「成功（7.4×）→ 正直な限界（底）→ しかし表現は本物の数論を学んでいた（再上昇・発見）→ fingerprint → positioning」という回復の弧になり、CICM聴衆（数論に最も反応する層）に向けて 結果パートの最後が NIG−φ 相関＝数論的発見で終わります。Slide 14 の fingerprint の一文が Slide 17 に直結するため、つなぎも1文で済みます。上のシミュレーションはこの並びで書いてあります。

提案2: 時間は現状ぎりぎり（試算 25:45)。Slide 7 を圧縮バッファに

100 wpm 想定で合計約25分45秒。本番は図の指差しや言い直しで膨らむので、 Slide 7（Training）を上記スクリプトの通り1分以内で流す運用にし、 押した場合は Slide 9 の「除外タグ3グループ」の各論を省略（スライドに書いてあるので口頭は1文)。 逆に余れば Backup 1（スケーリング）を Slide 15 の後に挿入。

提案3: Slide 10 の bridge で「3つの問い」を宣言する

結果4枚（11,12,15,16）が「Q1: 効くか → Q2: どこで → Q3: 厳密一致は」という問いの構造に ぶら下がると、聞き手が迷子にならない。スライド側にも小さく Q1/Q2/Q3 のラベルを振ると効果的。

細部

- Slide 2 冒頭は Moonshine の具体数（196,884 = 196,883 + 1）から入ると掴みが強い。
- Slide 6 では sin/cos を「位置エンコーディングの転用」と一言で説明（既にNote記載の通り)。
- Slide 16 の "measurement instrument" がこのトークで一番大事な一文。ゆっくり言う。